

CSCD602:

Advanced Software Engineering

Session 1: Introduction to Software Engineering

By
Solomon Mensah (PhD)



UNIVERSITY OF GHANA

Course Instructors

- Solomon Mensah (PhD)
 - Associate Professor, Dept. of Computer Science, UG
 - smensah03@ug.edu.gh
- Graduate Assistant
 - Benson
 - +233 (0) 54 694 9655

Agenda

- Recap
- What is Software Engineering?
- FAQs about Software Engineering
- SDLC
- AI in Software Engineering
- Emerging Trends

Learning Objectives

- Define software engineering and explain its importance
- Describe the Software Development Life Cycle (SDLC)
- Differentiate between major SDLC models
- Understand requirements engineering and design principles
- Discuss current trends in software engineering
- Analyze real-life software engineering case studies



Recap of Basic Software Engineering

- Definition of Software Engineering.
- Phases of the Software Development Life Cycle (SDLC):
 - Requirements Analysis
 - Design
 - Implementation
 - Testing
 - Deployment
 - Maintenance
- Importance of Software Engineering principles.

Introduction

- The economies of ALL developed nations are dependent on software
- More and more systems are software controlled
- Software engineering is concerned with theories, methods and tools for professional software development
- Software engineering is the systematic application of engineering approaches to the development of software.



Introduction

- Software costs often dominate system costs. The costs of software on a PC are often greater than the hardware cost
- Software costs more to maintain than it does to develop.
 - For systems with a long life, maintenance costs may be several times development costs
- Software engineering is concerned with cost-effective software development



What is Software Engineering?

- Software engineering is the systematic application of engineering principles to software development.
- It focuses on reliability, scalability, maintainability, and quality.
- Involves teamwork, documentation, testing, and maintenance.
- Aims to produce software efficiently within time and budget constraints.



Characteristics of Good Software

- Correctness – satisfies user requirements
- Reliability – performs consistently
- Efficiency – optimized performance and resources
- Maintainability – easy to modify and upgrade
- Usability – user-friendly interface and experience
- Security – protects data and systems



Importance of Software Engineering

- Reduces software failures and project risks
- Improves software quality and maintainability
- Supports large and complex systems
- Encourages teamwork and project management
- Ensures cost-effective software delivery



FAQs about software engineering

- What is software?
- What is software engineering?
- What is the difference between software engineering and computer science?
- What is the difference between software engineering and system engineering?
- What is a software process?
- What is a software process model?
- What is software design?



FAQs about software engineering

- What are the costs of software engineering?
- What are software engineering methods?
- What is CASE (Computer-Aided Software Engineering)?
- What are the attributes of good software?
- What are the key challenges facing software engineering?



What is software?

- Computer programs and associated documentation
- A software is a collection of data or computer instructions that tell the computer how to work.
- Software products may be developed for a particular customer or may be developed for a general market
- Software products may be
 - Generic - developed to be sold to a range of different customers
 - Bespoke (custom) - developed for a single customer according to his/her specification



What is Software engineering?

- Software engineering is an engineering discipline which is concerned with all aspects of software production or development
- Software engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available



What is the difference between software engineering and computer science?

- Computer science is concerned
 - the study of the principles and use of computers
 - with theory and fundamentals
- Software engineering is concerned with
 - the practicalities of developing and delivering useful software
 - the systematic application of engineering approaches to the development of software.
- Computer science theories are currently insufficient to act as a complete underpinning for software engineering



What is the difference between software engineering and system engineering?

- System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering
- Software engineering is part of this process
- System engineers are involved in system specification, architectural design, integration and deployment



What is a Software Process?

- A set of activities whose goal is the development or evolution of software
- Generic activities in all software processes are:
 - Specification - what the system should do and its development constraints
 - Development - production of the software system
 - Validation - checking that the software is what the customer wants
 - Evolution - changing the software in response to changing demands



What is a Software Process model?

- A simplified representation of a software process, presented from a specific perspective
- Examples of process perspectives are
 - Workflow perspective - sequence of activities
 - Data-flow perspective - information flow
 - Role/action perspective - who does what
- Generic process models
 - Agile
 - Waterfall (traditional approach)
 - Evolutionary development
 - Integration from reusable components



What is a Software Design?

- Software design is a process to transform user requirements into some suitable form, which helps the programmer in software coding and implementation.
- What (requirements) vs how (design)
- The design process converts the “what” into the “how”.



What is a Software Design?

- During this phase, tradeoffs need to be made, like the tradeoff between performance and resource consumption,
 - so that the application is optimized to meet the non-functional requirements.
- There can be no tradeoffs among functional requirements because they all need to be met.



Overview of the Software Engineering Lifecycle

- Planning and feasibility analysis
- Requirements gathering and analysis
- System design
- Implementation (coding)
- Testing and quality assurance
- Deployment
- Maintenance and evolution



Software Development Life Cycle (SDLC)

- A structured process for developing software systems
- Provides a roadmap from idea to deployment
- Helps manage complexity, time, cost, and quality
- Different SDLC models suit different project needs



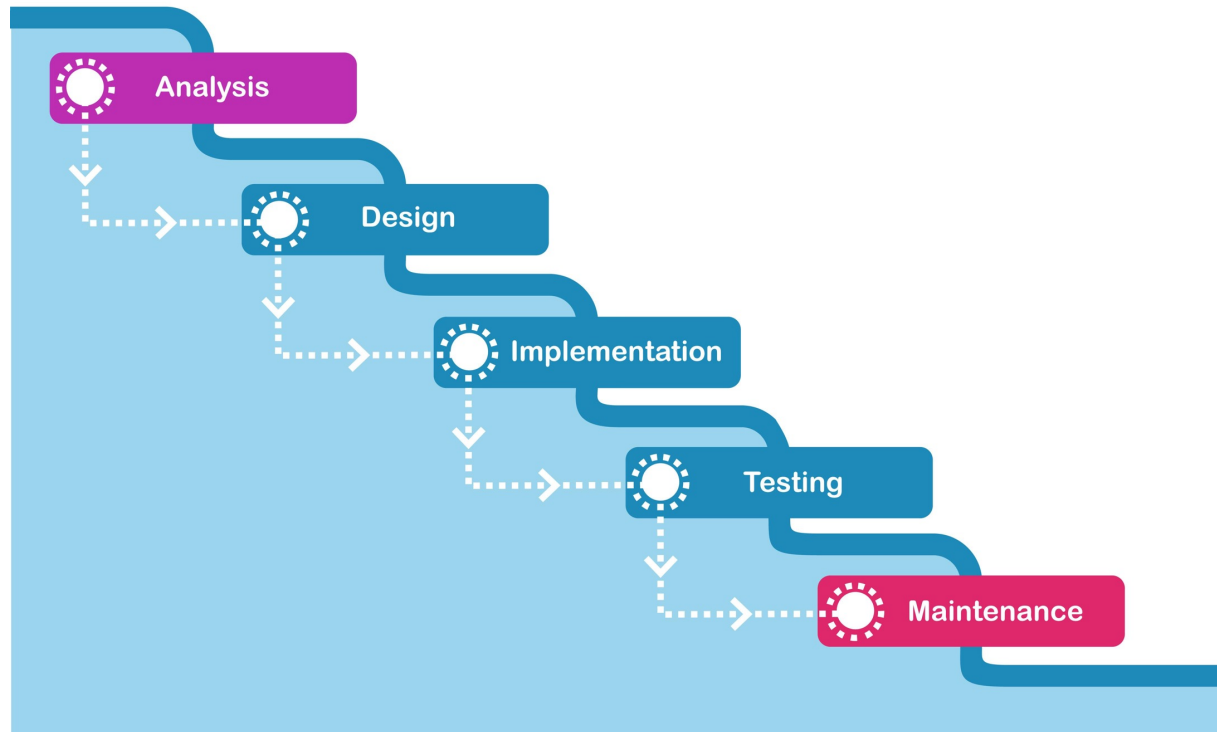
Waterfall Model

- Linear and sequential approach
- Each phase completed before the next begins
- Suitable for stable requirements
- Advantages: simplicity and clear documentation
- Disadvantages: inflexible and difficult to accommodate changes



Waterfall Model

WATERFALL

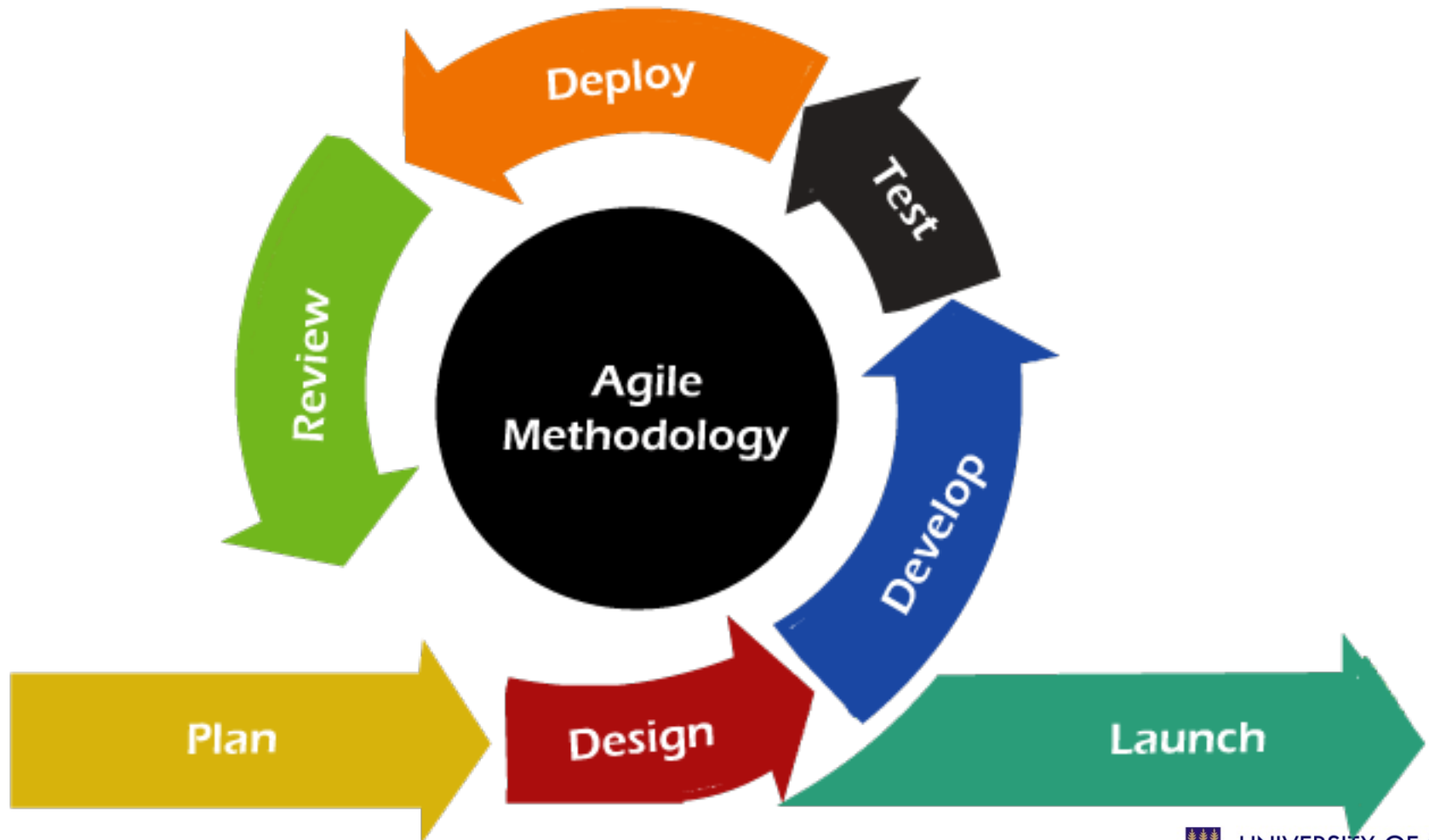


Agile Model

- Iterative and incremental development
- Focuses on customer collaboration and rapid delivery
- Uses short iterations called sprints
- Encourages adaptability and continuous feedback
- Popular frameworks: Scrum and Kanban



Agile Model



Spiral Model

- Combines iterative development with risk analysis
- Suitable for large and high-risk projects
- Focuses heavily on risk management
- Phases repeat in spirals until completion



SPIRAL



DevOps Approach

- Integrates software development and IT operations
- Promotes automation and continuous delivery
- Uses CI/CD pipelines
- Improves deployment speed and reliability



DevOps Approach

DevOps Engineer Role

Writing specifications and documentation



Infrastructure management



Performance assessment and monitoring



Cloud deployment and management



CI/CD management and automation



Assistance with DevOps culture adoption



Requirements Engineering

- Process of discovering, analyzing, documenting, and validating requirements
- Ensures software meets stakeholder expectations
- Requirements may be functional or non-functional
- Poor requirements often lead to project failure



Functional vs Non-Functional Requirements

- Functional Requirements: describe what the system should do
 - Examples: login, payment processing, report generation
- Non-Functional Requirements: describe system qualities
 - Examples: security, performance, scalability, usability



Requirements Engineering Process

- Elicitation – gathering requirements
- Analysis – resolving conflicts and prioritizing
- Specification – documenting requirements
- Validation – ensuring correctness and completeness
- Management – handling changes over time



Software Design Principles

- Abstraction – hiding unnecessary details
- Modularity – dividing systems into components
- Encapsulation – protecting data and behavior
- Separation of concerns – isolating functionalities
- Reusability – promoting reusable components



Software Engineering Principles

- Understand the problem before coding
- Design for change and maintainability
- Develop incrementally
- Focus on quality assurance
- Use documentation and standards
- Promote collaboration and communication



Software Quality Assurance

- Verification – building the product correctly
- Validation – building the correct product
- Code reviews and inspections
- Automated and manual testing
- Continuous monitoring and feedback



Software Testing Levels

- Unit Testing – testing individual components
- Integration Testing – testing interactions between modules
- System Testing – testing the entire system
- Acceptance Testing – validating against user requirements



Current Trends in Software Engineering

- Artificial Intelligence in software development
- Cloud computing and microservices
- Low-code and no-code platforms
- Cybersecurity engineering
- Internet of Things (IoT)
- Blockchain applications



Artificial Intelligence in Software Engineering

- AI-assisted coding tools improve productivity
- Predictive analytics for software defects
- Automated testing using machine learning
- Challenges include ethics, bias, and transparency



Cybersecurity in Software Engineering

- Security-by-design approach
- Threat modeling during development
- Secure coding practices
- Importance of data privacy regulations



Challenges in Software Engineering

- Changing requirements
- Budget and schedule overruns
- Software complexity
- Security vulnerabilities
- Team coordination issues
- Legacy system maintenance



Case Study I – Healthcare Information System

- Scenario: A national hospital system failed after deployment due to poor requirements gathering.
- Issues included missing stakeholder involvement and inadequate testing.
- Consequences: delayed patient services and data inconsistencies.
- Students should identify SDLC mistakes and propose improvements.



Case Study II – Mobile Banking Application

- A fintech company adopted Agile to rapidly develop a mobile banking app.
- Frequent user feedback improved usability and customer satisfaction.
- Challenges included security vulnerabilities and scalability concerns.
- Students should discuss Agile benefits and risk mitigation strategies.



Case Study III – Government E-Governance Platform

- Large-scale e-governance project using DevOps and cloud infrastructure.
- Project improved citizen service delivery.
- Faced challenges with data migration and cybersecurity threats.
- Students should evaluate the role of DevOps and cloud computing.



Group Discussion and Presentation Activity

- Form groups of 10-15 students
- Analyze assigned case study
- Identify software engineering challenges
- Recommend suitable SDLC model and solutions
- Prepare a 5-minute presentation with diagrams/examples



Suggested Discussion Questions

- What were the main causes of project success or failure?
- Which SDLC model would best fit the project?
- How could requirements engineering improve outcomes?
- What software engineering principles were violated or applied?
- How can modern trends improve the project?



Summary

- Software engineering applies disciplined methods to software development
- SDLC models provide structured development approaches
- Requirements engineering and design principles are critical
- Current trends continue to reshape software development
- Case studies help connect theory to practice



Emerging Trends

- Artificial Intelligence and Machine Learning in SE.
- Software for the Internet of Things (IoT).
- Blockchain Applications.
- Quantum Computing in Software Development.

Case Studies

- Real-world examples of advanced software engineering applications:
 - Amazon Web Services (AWS)
 - Google Search and Maps
 - Tesla's Autopilot

Thank you

